

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

**Лабораторна робота №9
з дисципліни
«Основи розробки програмного забезпечення мовою
програмування Python»
на тему
“Використання регулярних виразів для пошуку та заміни в рядках”**

Виконав:
студент групи ІМ-42
Жила Іван Дмитрович
номер варіанту: 8

Перевірив:
Фещенко К. Ю

Київ 2026

Мета роботи

Ознайомитися з основами регулярних виразів у мові програмування Python, вивчити їх синтаксис та навчитися застосовувати регулярні вирази для пошуку, перевірки та заміни фрагментів тексту з використанням модуля *re*.

Короткі теоретичні відомості:

1. Поняття регулярних виразів

Регулярні вирази (regex) — це спеціальні шаблони, які описують множину рядків і дозволяють виконувати пошук, перевірку (валідацію) та заміну текстових даних у програмі. Вони широко застосовуються для перевірки коректності введення даних (email, телефони, IP-адреси), аналізу великих текстових файлів, обробки логів та автоматизованої заміни текстових фрагментів.

2. Модуль *re* у Python

Для роботи з регулярними виразами в Python використовується стандартний вбудований модуль *re*. Основними функціями цього модуля є:

- *re.search(pattern, string)* — здійснює пошук лише першого входження шаблону в рядку.
- *re.findall(pattern, string)* — знаходить усі входження шаблону в рядку та повертає їх у вигляді списку.
- *re.match(pattern, string)* — перевіряє, чи починається рядок безпосередньо з заданого шаблону.
- *re.sub(pattern, repl, string)* — замінює всі знайдені входження шаблону на вказаний рядок заміник (*repl*).

3. Основні спеціальні символи та квантифікатори

Для побудови шаблонів використовуються спеціальні метасимволи та класи символів:

- *.* — означає будь-який одиничний символ, окрім символу нового рядка.
- *\d* — будь-яка цифра від 0 до 9 (еквівалентно *[0-9]*).
- *\w* — будь-який алфавітно-цифровий символ слова (буква, цифра або знак підкреслення *_*).
- *\s* — пробільний символ (пробіл, табуляція, перенос рядка).

- ^ та \$ — фіксують початок і кінець рядка відповідно.

Для встановлення кількості повторень символів застосовують квантифікатори: * (0 або більше разів), + (1 або більше разів), ? (0 або 1 повторення), а також {m,n} (від \$m\$ до \$n\$ повторень).

4. Особливості синтаксису в Python

При створенні регулярних виразів у Python обов'язковою практикою є використання сирих рядків (raw strings) із префіксом r перед лапками (наприклад, r'\d+'). Це запобігає інтерпретації символу зворотного слешу (\) як екрануючого символу самої мови Python, що дозволяє передавати шаблони до модуля re у незмінному вигляді.

Загальне завдання:

Написати програму, яка:

- вводить початкові дані (з файлу або з клавіатури);
- формує результат з використанням регулярних виразів;
- виводить результат на екран або зберігає у файл.

Індивідуальне завдання — Реалізувати програму для заміни всіх дат у форматі dd-mm-уууу на уууу mm-dd. (8 варіант)

Текст програми:

```
# Лабораторна 9

# Тема: Використання регулярних виразів для пошуку та заміни в рядках

# Загальне завдання: Написати програму, яка вводить початкові дані,
формує результат
# з використанням регулярних виразів та виводить його на екран або
зберігає у файл.

# Варіант 8: Реалізувати програму для заміни всіх дат у форматі dd-mm-
уууу на уууу mm-dd.
```

```

import re
import os

# Функція для відображення вмісту файлу в консолі
def print_file_content(filename, description):
    print(f"\nВміст файлу: {description} ({filename})")
    if os.path.exists(filename):
        with open(filename, 'r', encoding='utf-8') as f:
            print(f.read())
    else:
        print("Файл не знайдено.")
    print("-" * 50)

# Функція для виконання загального завдання
# (Пошук усіх цифр/чисел у тексті)
def process_general_task(input_text, output_file):
    print("\nПошук чисел у тексті")

    # findall – повертає список усіх збігів за шаблоном
    # \d+ – означає одну або більше цифр
    numbers = re.findall(r'\d+', input_text)

    result_str = f"Оригінальний текст: {input_text}\nЗнайдені числа: {'',
''.join(numbers)}"
    print(result_str)

    # Зберігаємо результат у файл
    with open(output_file, 'w', encoding='utf-8') as f:
        f.write(result_str)
    print(f"Результат загального завдання збережено у '{output_file}'")

# Функція для виконання індивідуального завдання
# Заміна формату дати за допомогою re.sub()
def process_individual_task(input_text, output_file):
    print("\nФорматування дат")

    # Шаблон для dd-mm-уууу: дві цифри, дефіс, дві цифри, дефіс, чотири
цифри
    # Використовуємо групування (), щоб потім змінити їх місцями
    date_pattern = r'(\d{2})-(\d{2})-(\d{4})'

    # re.sub(pattern, repl, string) замінює всі входження за шаблоном
    # \3-\2-\1 – це посилання на групи (рік-місяць-день)
    processed_text = re.sub(date_pattern, r'\3-\2-\1', input_text)

    print(f"Текст до обробки: {input_text}")
    print(f"Текст після заміни дат: {processed_text}")

    # Записуємо оброблений текст у файл
    with open(output_file, 'w', encoding='utf-8') as f:

```

```

        f.write(processed_text)
    print(f"Результат індивідуального завдання збережено у '{output_file}''")

def main():
    # Початкові дані
    raw_text = "Сьогодні 25-10-2023, а наступна зустріч відбудеться 05-11-2023."

    # Виконання загального завдання (пошук чисел)
    process_general_task(raw_text, 'general_results.txt')

    # Виконання заміни формату дат
    process_individual_task(raw_text, 'formatted_dates.txt')

    # Перевірка вмісту створених файлів
    print_file_content('general_results.txt', "Результати пошуку чисел")
    print_file_content('formatted_dates.txt', "Текст із виправленими датами")

if __name__ == "__main__":
    main()

```

Скріншоти:

```

Пошук чисел у тексті
Оригінальний текст: Сьогодні 25-10-2023, а наступна зустріч відбудеться 05-11-2023.
Знайдені числа: 25, 10, 2023, 05, 11, 2023
Результат загального завдання збережено у 'general_results.txt'

Форматування дат
Текст до обробки: Сьогодні 25-10-2023, а наступна зустріч відбудеться 05-11-2023.
Текст після заміни дат: Сьогодні 2023-10-25, а наступна зустріч відбудеться 2023-11-05.
Результат індивідуального завдання збережено у 'formatted_dates.txt'

Вміст файлу: Результати пошуку чисел (general_results.txt)
Оригінальний текст: Сьогодні 25-10-2023, а наступна зустріч відбудеться 05-11-2023.
Знайдені числа: 25, 10, 2023, 05, 11, 2023
-----

Вміст файлу: Текст із виправленими датами (formatted_dates.txt)
Сьогодні 2023-10-25, а наступна зустріч відбудеться 2023-11-05.
-----

```

Висновок:

Під час виконання лабораторної роботи я ознайомився з основами використання регулярних виразів у мові програмування Python. Набув практичних навичок роботи зі стандартним модулем `re` та його основними функціями для ефективного пошуку та маніпуляцій із текстовими даними.

У ході роботи я успішно реалізував індивідуальне завдання, розробивши алгоритм для автоматичного переформатування дат у тексті з формату `dd-mm-uuuu` на `uuuu-mm-dd` за допомогою функції `re.sub()` та механізму групування символів. Також я опанував використання сирих рядків (`raw strings`) для коректної передачі шаблонів інтерпретатору. Виконання цієї лабораторної роботи підтвердило, що використання регулярних виразів значно спрощує та прискорює процес валідації, аналізу та редагування складних текстових структур.